

```
# %% Imports and File Paths
import pandas as pd
import numpy as np

# Tells NumPy to use the legacy print format, which prints plain numbers instead of
# verbose types (e.g. '3.0' instead of 'np.float64(3.0)')
np.set_printoptions(legacy="1.25")

# File paths to the toy_net data
# (update these if your data files are in a different location).
node_file = "data/toy_net/nodes.csv"
link_file = "data/toy_net/net.csv"
demand_file = "data/toy_net/trips.csv"

# %% Load the Data - Task 1

# Load nodes.csv into df_nodes and set 'node' as the index.
df_nodes = pd.read_csv(node_file, sep=",")
df_nodes.set_index("node", inplace=True, drop=False)

# Load net.csv into df_links and set ['init_node', 'term_node'] as a MultiIndex.
df_links = pd.read_csv(link_file, sep=",")
df_links.set_index(["init_node", "term_node"], inplace=True, drop=False)

# Load trips.csv into df_ods and set ['orig', 'dest'] as a MultiIndex.
df_ods = pd.read_csv(demand_file, sep=",")
df_ods.set_index(["orig", "dest"], inplace=True, drop=False)

print("--- Network Data ---")
print("df_nodes:")
print(df_nodes)
print("\ndf_links:")
print(df_links)
print("\ndf_ods:")
print(df_ods)

# %% Initialisation

# Construct the forward star dictionary to store the downstream nodes for each node.
forward_star = {}

# Loop through all nodes in df_nodes to initialise forward_star with empty lists.
for row in df_nodes.itertuples(index=False):
    forward_star[row.node] = []

# Loop through all links in df_links to populate the forward_star dictionary.
for row in df_links.itertuples(index=False):
    forward_star[row.init_node].append(row.term_node)

print("\nforward_star:", forward_star)

# Create the 'cost' column to store link travel time
# Initialise the values to 'free_flow_time'
df_links["cost"] = df_links["free_flow_time"]

# Create the 'flow' column to store link flow values
# Initialise the values to 0.0
df_links["flow"] = 0.0

# Create the 'shortest_path_time' column in df_ods to store
# the shortest path travel time for each OD pair
df_ods["shortest_path_time"] = 0.0

# Create the 'shortest_path' column in df_ods to store the list of node IDs
# for the shortest path for each OD pair;
# Initialise with 'None' to store 'object' data type, such as lists or strings
df_ods["shortest_path"] = None

print("\n--- Prepared for Assignment ---")
print("\nforward_star:", forward_star)
print("\ndf_links:")
print(df_links)
print("\ndf_ods:")
print(df_ods)

# %% Find Shortest Paths for All OD-Pairs ㄟㄟ Task 2
import w5_sol_functions as fn

# Loop over unique origin nodes that appear in df_ods.
for orig_node in df_ods["orig"].unique():
    L, q, odpath = fn.shortestPath(orig_node, df_nodes, df_links, forward_star)

    # Loop over destination nodes for this origin.
    for dest_node in df_ods[df_ods["orig"] == orig_node]["dest"]:
        od_index = (orig_node, dest_node)
        # Ensure that od_index is in odpath before accessing it to avoid KeyError
        if od_index in odpath:
            df_ods.at[od_index, "shortest_path_time"] = L[dest_node]
            df_ods.at[od_index, "shortest_path"] = odpath[od_index]

print("\n--- OD Shortest Paths ---")
print(df_ods)

# %% Assign Demand to Shortest Paths and Update Link Flows ㄟㄟ Task 3

# Loop over all OD pairs in df_ods and accumulate each OD demand onto
# every link along the shortest path for that OD pair.
for row in df_ods.itertuples(index=False):
    demand = row.demand
    path = row.shortest_path
    for i in range(len(path) - 1):
        link_index = (path[i], path[i + 1])
        df_links.at[link_index, "flow"] += demand

print("\n--- Link Flows after Assignment ---")
print(df_links[["flow"]])

# %% Update Link Costs using BPR ㄟㄟ Task 4

# Loop over all links in df_links and update the 'cost' column using the BPR function
for link in df_links.itertuples(index=True):
    # Compute travel time using the BPR function
    t = link.free_flow_time * (1 + link.b * pow(link.flow / link.capacity, link.power))
    # Update link cost
    df_links.at[link.Index, "cost"] = t

print("\n--- Link Flows and Costs after BPR Update ---")
print(df_links[["capacity", "flow", "free_flow_time", "cost"]])

# %%
```